EuroSAT Land Use Classification - Demo Script

Module: NB6057CEM | Target Time: 5 to 7 Minutes | Coverage: Notebooks 01 to 08

1. Recommended Time Management

Section	Notebooks Covered	Allocated Time	Importance
Part 1: Data & Split	NB 01 (Download) & NB 02 (EDA)	0:00 - 0:45 (45s)	Important (T1: 8 marks)
Part 2: NumPy MLP	NB 03 (Hand-coded Backprop)	0:45 - 2:00 (1m 15s)	CRITICAL (T2: 12 marks)
Part 3: Baseline & Models	NB 04 (HOG), NB 05 (MLP), NB 06 (CNN)	2:00 - 3:30 (1m 30s)	High (T3: 14 marks)
Part 4: Experiments	NB 07 (Evaluation & Noise Stress)	3:30 - 4:45 (1m 15s)	CRITICAL (T4: 14 marks)
Part 5: Live Inference	NB 08 (Recommendation & Demo)	4:45 - 6:00 (1m 15s)	High (T5/T6: 12 marks)

Part 1: Introduction, Data Acquisition & Split (45 seconds)

Action on Screen: Open Notebook 01 (01_data_download.ipynb) and then Notebook 02 (02_eda_split.ipynb)

Hello everyone. Today I will demonstrate my entire EuroSAT Land Use Classification coursework.

In Notebook 1, I downloaded the EuroSAT RGB dataset containing 27,000 satellite images across 10 classes. Each image has 64 by 64 RGB pixels. I saved the dataset as an NPZ archive to Google Drive for fast local loading.

In Notebook 2, I performed a data audit. Zero corrupt images and zero exact duplicates were found. I created a stratified 70% train, 15% validation, and 15% test split. To strictly prevent data leakage, channel mean and standard deviation were calculated only from the training set.

Part 2: NumPy Multi-Layer Perceptron from Scratch (1 minute 15 seconds)

Action on Screen: Open Notebook 03 (03_numpy_mlp.ipynb) - Show Python class and gradient check table

In Notebook 3, I implemented an MLP completely from scratch using NumPy without PyTorch autograd.

I trained it on a 4-class subset resized to 16 by 16 grayscale. The architecture is 256 inputs, one hidden layer of 32 units with tanh activation, and 4 softmax outputs.

I manually derived the backward pass using the chain rule to update weights and biases. To rigorously verify my math, I implemented a numerical gradient check with central differences. All relative errors are under 1e-7, proving the analytical gradients are 100% correct. I also overfitted a tiny 10-image batch to confirm training loss reaches zero.

Part 3: Baseline & Deep Learning Models (1 minute 30 seconds)

Action on Screen: Open Notebooks 04, 05, and 06 (04_baseline_hog_lr, 05_mlp_pytorch, 06_cnn_pytorch)

Next, I trained three comparative models on all 10 classes:

In Notebook 4, the Baseline extracts hand-crafted HOG edge orientations and color histograms with Logistic Regression. It reached 0.73 Test Macro-F1 with 17,880 parameters.

In Notebook 5, the PyTorch MLP has two hidden layers of 512 and 256 neurons. It has 6.4 million parameters but only achieved 0.51 Macro-F1. Flat linear layers ignore spatial 2D relationships, leading to high parameters and poor generalization.

In Notebook 6, the PyTorch CNN uses 3 convolutional blocks with 32, 64, and 128 channels, batch normalization, and data augmentation. It achieved 0.91 Macro-F1 and 93% accuracy with only 94,730 parameters - 68 times fewer parameters than the MLP.

Part 4: Comparative Evaluation & Noise Stress Test (1 minute 15 seconds)

Action on Screen: Open Notebook 07 (07_experiments_evaluation.ipynb) - Show summary table, confusion matrix & noise plot

In Notebook 7, all models were evaluated across 3 random seeds:

The summary confirms CNN is the top model with 0.9103 Test Macro-F1 versus 0.5100 for MLP and 0.7324 for Baseline. Individual classes like Forest, Residential, and SeaLake reached 97% to 98% F1. Misclassifications mainly occurred between visually similar terrain like River and Highway.

I also conducted a Gaussian noise stress test. As noise sigma increased from 0.0 to 0.20, Macro-F1 dropped sharply from 0.92 to 0.16. This reveals that the CNN is sensitive to atmospheric sensor noise and would require denoising in a production satellite pipeline.

Part 5: Live Inference Pipeline & Edge Case Demo (1 minute 15 seconds)

Action on Screen: Open Notebook 08 (08_recommendation_inference.ipynb) - Run or highlight the prediction output cells

Finally, in Notebook 8, I benchmarked inference latency and deployed the prediction pipeline:

Inference takes only 0.73 milliseconds per batch of 64 on GPU, proving it is ready for real-time applications.

Here is the live inference demonstration:
1. Valid Satellite Image: Correctly classified as PermanentCrop with 99.2% confidence.
2. Invalid Text File: The pipeline safely catches the error and reports an unreadable file.
3. Out-of-Scope White Noise: The network predicts Highway with 88.8% confidence. This demonstrates an essential academic finding: Softmax confidence is imperfect for out-of-distribution detection, as neural networks can be confidently wrong on synthetic noise.

In conclusion, I recommend the CNN for deployment due to high accuracy, low parameters, and sub-millisecond latency. Thank you.

3. Examiner Q&A Cheat Sheet (Short Answers)

Q1: Why use Macro-F1 instead of Accuracy?
A: Macro-F1 evaluates all classes equally. Accuracy can hide poor performance on smaller minority classes.

Q2: Why does CNN outperform MLP with 68x fewer parameters?
A: CNN has spatial inductive bias: local receptive fields and shared convolutional weights capture spatial features regardless of pixel location.

Q3: What is the biological analogy of an artificial neuron?
A: Inputs represent dendrites, weights represent synaptic strengths, summation is the cell soma membrane potential, and activation is the axon firing action.

Q4: Why was gradient check done with tanh instead of ReLU?
A: ReLU has a non-differentiable kink at zero which causes finite difference checks to fail. Tanh is smooth and differentiable everywhere.

Q5: What was the main finding of the noise stress test?
A: It proved high clean test accuracy does not guarantee robustness against sensor degradation; performance dropped from 0.92 to 0.16 at sigma=0.20.